

quallmer: A toolkit for qualitative analysis with large language models

Seraphine F. Maerz^{1*} and Kenneth Benoit^{2*}

¹ University of Melbourne, Australia ² Singapore Management University, Singapore * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 06 May 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

quallmer is an R package that enables researchers to apply large language model (LLM)-assisted qualitative coding to texts, images, PDFs, audio, and tabular data while maintaining the rigorous standards of transparency and traceability essential to qualitative research. The package provides a complete workflow for: (1) flexibly defining comprehensive codebooks tailored to specific research questions and data types; (2) applying LLM-based analyses; (3) replicating analyses across different models and settings; (4) comparing results using established inter-rater reliability metrics and validating outputs against human-coded gold standards; and (5) documenting complete audit trails that capture the full decision history of the workflow, including model parameters, timestamps, and parent-child relationships across analyses. This audit trail functionality operationalizes the trustworthiness criteria established by Lincoln & Guba (1985), meeting their standards of credibility, transferability, dependability, and confirmability in LLM-assisted qualitative research. Built on the *ellmer* package (Wickham et al., 2025), *quallmer* supports a wide range of LLM providers—including OpenAI, Anthropic Claude, Google Gemini, Mistral, and local models via Ollama—giving researchers flexibility in their choice of models.

Statement of need

Qualitative content analysis requires systematic coding of textual or visual material according to well-defined coding schemes (Krippendorff, 2019). Researchers applying these methods face a fundamental challenge: manual coding is rigorous but does not scale to large corpora. LLMs offer unprecedented capacity to process qualitative data at scale, but their integration into rigorous research workflows is non-trivial.

Existing AI-assisted tools fall short in different ways. Proprietary qualitative data analysis software such as NVivo (2026), MAXQDA (2026), and QDAMiner (2026) have added AI-assisted coding features, but even the most transparent of these (QDAMiner, which allows researchers to select models and edit prompts) operates entirely through a graphical interface with no code-based workflow, no comprehensive audit trail, and no built-in reliability assessment framework. Researchers who instead write custom LLM code gain flexibility but must build substantial infrastructure from scratch—managing codebooks, tracking provenance, computing reliability metrics, and documenting workflows—diverting effort from substantive research questions (e.g., Fang et al., 2026). General-purpose R packages for LLM access (Gruber & Weber, 2024; Lin & Safi, 2025; Wickham et al., 2025) and text analysis (Benoit et al., 2018) do not address the specific methodological requirements of trustworthy qualitative research.

quallmer fills this gap. The package is designed for social scientists, qualitative researchers, and computational social scientists who need to apply LLM-assisted qualitative coding at scale while

41 maintaining methodological rigor. Researchers can use *quallmer* without deep programming
42 expertise, lowering barriers to adoption while preserving the transparency, reliability, and
43 reproducibility that rigorous qualitative work demands.

44 State of the field

45 The dominant tools for computer-assisted qualitative data analysis are commercial interactive
46 software packages: NVivo (2026), MAXQDA (2026), and QDAMiner (2026) are widely used
47 in social science research. These tools offer graphical interfaces that make qualitative coding
48 accessible, but their workflows are inherently interactive and non-reproducible: analyses cannot
49 be executed as code and results cannot be straightforwardly replicated by other researchers.
50 All three have added AI-assisted coding features at various points, but none fully addresses
51 these fundamental limitations. In addition, none of these tools is open-source.

52 NVivo has offered machine-learning-based auto-coding since version 11 (2015), including
53 sentiment detection, theme identification, and pattern-based coding. Of these, only
54 pattern-based coding—which extends a researcher’s own manual coding to new material—is
55 customisable; sentiment and theme detection use fixed categories and noun-phrase frequency
56 analysis respectively. NVivo 15 (2024) added a generative AI Assistant with more flexible
57 capabilities, including AI-suggested subcodes within the researcher’s existing framework, but
58 these remain GUI-driven with no programmatic interface. MAXQDA introduced AI-assisted
59 coding in 2024 (version 24.5), and does provide some transparency: each AI-coded segment is
60 accompanied by an explanation of the coding rationale, and AI-generated codes are visually
61 distinguished from manual codes. However, MAXQDA does not disclose which model or
62 version powers its AI, and provides no structured audit log or mechanism for exact replication.
63 QDAMiner 2025 takes a notably different approach, allowing researchers to select their own
64 LLM provider and model, edit the prompts used for coding, and monitor token usage and costs
65 per project—the most transparent AI implementation among major CAQDAS tools. Yet like
66 its competitors, QDAMiner operates entirely through a graphical interface with no code-based
67 workflow.

68 Lincoln & Guba (1985) established credibility, transferability, dependability, and confirmability
69 as the essential trustworthiness criteria for rigorous qualitative research, broadly corresponding
70 to internal validity, external validity, reliability, and objectivity in conventional inquiry (King et
71 al., 1994; Seale, 1999). Meeting these standards requires detailed *audit trails* documenting
72 analytical decisions, coding instructions, model parameters, and the rationale behind key
73 choices. Lincoln and Guba (1985, pp. 319–320) describe six features such trails should provide:
74 raw data, data reduction products, data reconstruction products, process notes, information
75 on intentions and dispositions, and instrument development information. None of the major
76 CAQDAS tools provides a mechanism for generating such comprehensive audit trails. While
77 MAXQDA and QDAMiner offer partial documentation of AI coding decisions, no tool records
78 the full provenance chain—including model identifiers, parameter settings, timestamps, and
79 parent-child relationships across analyses—needed for systematic replication. The stochastic
80 nature of LLM outputs compounds this problem: neither NVivo nor MAXQDA discloses or
81 allows researchers to lock the underlying model version, so re-running an analysis may produce
82 different results for reasons outside the researcher’s control.

83 Among R packages, *ellmer* (Wickham et al., 2025), *ollamar* (Lin & Safi, 2025), and *rollama*
84 (Gruber & Weber, 2024) provide flexible LLM interfaces but offer no qualitative methodology
85 support; *quanteda* (Benoit et al., 2018) supports quantitative text analysis but not AI-assisted
86 qualitative coding. *quallmer* builds on *ellmer* for LLM access rather than reimplementing that
87 layer. As open-source software, it allows full inspection of its implementation, and its unique
88 scholarly contribution is the codebook, audit trail, and reliability assessment framework that
89 operationalizes Lincoln & Guba (1985)’s trustworthiness criteria directly in code.

Software design

quallmer implements a structured five-step workflow through five core functions, summarised in ???. Each function addresses specific requirements of trustworthy qualitative research as established by Lincoln & Guba (1985) and Krippendorff (2019).

Table 1: Core *quallmer* functions and their role in the five-step workflow.

Step	Function(s)	Purpose
1. Define codebook	<code>qlm_codebook()</code>	Create reusable coding schemes with instructions and structured output schema
2. Code data	<code>qlm_code()</code> , <code>qlm_segment()</code>	Apply a codebook to texts, images, PDFs, audio, or tabular data using any supported LLM; or segment texts into thematic units and optionally apply codes in the same pass
3. Replicate	<code>qlm_replicate()</code>	Re-run coding with different models or parameter settings, preserving provenance chains
4. Compare & validate	<code>qlm_compare()</code> , <code>qlm_validate()</code>	Compute inter-rater reliability metrics including unitizing alpha; benchmark against gold standards
5. Audit trail	<code>qlm_trail()</code>	Generate complete audit documentation and an executable replication report

For analyses requiring variable-length text units, `qlm_segment()` uses an LLM to segment texts into thematic or conceptual units before coding—useful for aspect-based sentiment analysis, quasi-sentence segmentation, or topic-based chunking. When comparing segmented outputs, `qlm_compare()` computes Krippendorff’s alpha for unitizing (Krippendorff, 2019, sec. 12.6), which jointly measures agreement on both where boundaries fall and how segments are coded. The function also reports unitization-conditional alpha, isolating coding disagreement from boundary disagreement, as well as per-code reliability metrics for diagnosing which categories are applied consistently.

The package is built on the *ellmer* framework (Wickham et al., 2025), which provides a flexible interface to a wide range of LLM providers. This design ensures researchers can choose models that fit their research questions and resource constraints without *quallmer* managing LLM connectivity. The architecture is modular and extensible, allowing customization of codebooks, coding procedures, and validation metrics while ensuring all analyses are fully documented.

quallmer evolved from an earlier implementation (a draft R package entitled *quanteda.llm* (Maerz & Benoit, 2025)) that integrated LLM capabilities with the *quanteda* framework (Benoit et al., 2018). Feedback from workshops and training sessions with social scientists shaped key design decisions: the highly customizable codebook definition, the built-in reliability metrics, and especially the audit trail functionality. A companion package, *quallmer.app* (also on CRAN), provides a Shiny application for manual coding, review of LLM-assisted coding, and agreement metric computation, bridging automated and human-in-the-loop workflows.

For analyses requiring variable-length text units, `qlm_segment()` uses an LLM to segment texts into thematic or conceptual units—useful for aspect-based sentiment analysis, quasi-sentence segmentation, or topic-based chunking. Codes can be applied to each segment in the same pass by including them in the codebook schema, making a separate `qlm_code()` call unnecessary. When comparing segmented outputs, `qlm_compare()` computes Krippendorff's alpha for unitizing (Krippendorff, 2019, sec. 12.6), which jointly measures agreement on both segmentation boundaries as well as how these are coded, even when two sets of coded segments differ in length and segment boundaries. This is done by implementing of Krippendorff (2019)'s unitization-conditional unitization-conditional $cu\alpha[n]$ for isolating coding disagreement from boundary disagreement overall, and by category through $(k)u\alpha[n]$. To our knowledge, no other R package has implemented these reliability statistics.

Worked example: coding political speeches

To illustrate the workflow, consider analyzing political rhetoric in speeches. The complete tutorial is at <https://quallmer.github.io/quallmer/>.

Step 1: Codebook definition. `qlm_codebook()` defines a reusable coding scheme specifying instructions for the LLM and the expected output structure using *ellmer* type specifications:

```
library(quallmer)

codebook_ideology <- qlm_codebook(
  name = "Liberal-illiberal rhetoric",
  instructions = "Analyze the rhetorical style of this political speech.
    ILLIBERAL rhetoric (negative scores): nationalism, paternalism, traditionalism.
    LIBERAL rhetoric (positive scores): individual rights, tolerance, civil liberties.
    Score 0 = neutral/mixed.",
  schema = type_object(
    score = type_integer("Score from -10 (illiberal) to +10 (liberal)"),
    explanation = type_string("Brief explanation of the score")
  )
)
```

Step 2: Data coding. `qlm_code()` applies the codebook to texts using any *ellmer*-supported LLM, returning a `qlm_coded` object with coding results and full metadata (timestamps, model identifiers, codebook used):

```
coded_speeches <- qlm_code(
  speech_corpus,           # character vector or quanteda corpus
  codebook = codebook_ideology, # qlm_codebook object
  model = "openai/gpt-4o-mini", # provider and model
  name = "ideology_coding_run1" # user-assigned model name
)
```

Step 3: Replication. `qlm_replicate()` re-runs coding with different models or parameter settings, maintaining provenance chains linking replicated results to their parent analyses:

```
coded_claude <- qlm_replicate(coded_speeches, model = "anthropic/claude-3-5-sonnet")
coded_temp07 <- qlm_replicate(coded_speeches, temperature = 0.7)
```

Step 4: Comparison and validation. `qlm_compare()` computes inter-rater reliability tailored to measurement level: Krippendorff's alpha, Cohen's/Fleiss' kappa, and percent agreement for nominal data; Krippendorff's alpha, weighted kappa, Kendall's W, Spearman's rho, and percent agreement for ordinal data; and Krippendorff's alpha, ICC, Pearson's r, and percent agreement for interval/ratio data. For segmented corpora, it additionally computes Krippendorff's unitizing alpha (boundary agreement), unitizing alpha with codes (boundary and coding agreement jointly), coding-conditional alpha (coding agreement given shared boundaries), and per-category

unitizing alpha. `qlm_validate()` benchmarks against human-coded gold standards, computing accuracy, precision, recall, F1, and Cohen's kappa for nominal data; Spearman's rho, Kendall's tau, and MAE for ordinal data; and ICC, Pearson's r, MAE, and RMSE for interval data:

```
comparison <- qlm_compare(coded_speeches, coded_claude, by = "score", level = "interval")
validation <- qlm_validate(coded_speeches, gold_standard, by = "score", level = "interval")
```

Step 5: Audit trail. `qlm_trail()` generates a complete audit package following Lincoln & Guba (1985), including an `.rds` trail object and an executable Quarto replication report:

```
qlm_trail(coded_speeches, path = "replication_materials")
```

Research impact statement

quallmer is available on CRAN and actively maintained at <https://github.com/quallmer/quallmer>. The companion *quallmer.app* package is also on CRAN. Applications have included large-scale content analysis of political documents, media coverage, and sentiment analysis among others. The package has been tested and applied in ongoing research projects by the authors and collaborating researchers (including Andrew Walter and Dean Schafer), with peer-reviewed publications using *quallmer* in preparation. Its theoretical grounding in Lincoln & Guba (1985)'s trustworthiness framework, combined with integration into the established *ellmer* and *quanteda* R ecosystems, positions *quallmer* as methodological infrastructure for the growing community of computational social science researchers applying LLMs to qualitative data.

AI usage disclosure

In developing *quallmer*, we used AI coding assistants, specifically Anthropic's Claude (via Claude Code) and GitHub Copilot, for code refactoring, documentation drafting, and test development. All AI-generated contributions were reviewed, validated, and revised by the human authors, who retained full responsibility for all design decisions, the intellectual framing of the package, and the conceptual framework grounding the audit trail functionality in established qualitative methodology. The use of AI tools accelerated implementation but did not substitute for the substantive scholarly decisions that define the package's contribution. This paper was drafted with AI assistance; all content was reviewed and substantially revised by the authors.

Acknowledgements

We gratefully acknowledge feedback from participants in our workshops and training sessions, whose experiences and suggestions substantially shaped the package design. We also thank Andrew Walter and Dean Schafer for testing early versions of the package, including *quallmer.app*.

References

- Benoit, K., Watanabe, K., Wang, H., Nulty, P., Obeng, A., Müller, S., & Matsuo, A. (2018). Quanteda: An r package for the quantitative analysis of textual data. *Journal of Open Source Software*, 3(30), 774. <https://doi.org/10.21105/joss.00774>
- Fang, Q., Garcia Bernardo, J., & Kesteren, E.-J. van. (2026). *A Methodological Guide on Using Large Language Models for Text Annotation in the Social Sciences and Humanities with Python and R*. OSF preprint. https://osf.io/preprints/socarxiv/v4eq6_v2
- Gruber, J. B., & Weber, M. (2024). *rollama: An R package for using generative large language models through Ollama*. 1–4. <http://arxiv.org/abs/2404.07654>

- 181 King, G., Keohane, R. O., & Verba, S. (1994). *Designing social inquiry: Scientific inference in*
182 *qualitative research*. Princeton University Press.
- 183 Krippendorff, K. (2019). *Content analysis: An introduction to its methodology* (4th ed.).
184 SAGE Publications.
- 185 Lin, H., & Safi, T. (2025). ollamar: An R package for running large language models. *Journal*
186 *of Open Source Software*, 10(105), 7211. <https://doi.org/10.21105/joss.07211>
- 187 Lincoln, Y. S., & Guba, E. G. (1985). *Naturalistic inquiry*. SAGE Publications.
- 188 Maerz, S. F., & Benoit, K. (2025). *Quanteda.llm: LLM support for quanteda*. <https://github.com/quanteda/quanteda.llm>
- 190 MAXQDA. (2026). *Discover what is possible with AI assist*. <https://www.maxqda.com/products/ai-assist>
- 192 NVivo. (2026). *NVivo automated coding with AI*. <https://lumivero.com/resources/product-tutorials/nvivo-automated-coding-with-ai/>
- 194 Provalis Research. (2026). *QDAMiner qualitative data analysis software*. <https://provalisresearch.com/products/qualitative-data-analysis-software/>
- 196 Seale, C. (1999). Guiding ideals. In *The quality of qualitative research* (pp. 32–50). SAGE
197 Publications.
- 198 Wickham, H., Cheng, J., Jacobs, A., Aden-Buie, G., & Schloerke, B. (2025). *Ellmer: Chat*
199 *with large language models*. <https://ellmer.tidyverse.org>